

Veritas: Dynamic Evaluation Framework for RAG Systems' Defense Against Prompt Injection Attacks

Ankita John

Dept. of Computing and Software Systems
University of Washington Bothell
 Bothell, USA
 amj59@uw.edu

Geethapriya Thamilarasu

Dept. of Computing and Software Systems
University of Washington Bothell
 Bothell, USA
 geetha@uw.edu

Abstract—Prompt injection poses a significant security risk to Retrieval-Augmented Generation (RAG) systems, enabling adversaries to embed malicious instructions within retrieved documents and hijack model behavior to exfiltrate sensitive information or execute unauthorized actions. This work presents a modular dynamic evaluation environment for systematically testing and comparing defense mechanisms against prompt injection attacks in RAG architectures. The framework simulates diverse injection scenarios targeting the retrieval pipeline, integrates optional mitigation strategies such as input filtering, prompt rewriting, and retrieval-aware defenses, and automatically logs model behavior to assess attack success. By varying retrieval parameters and quantifying defense robustness across diverse attack verticals and RAG configurations, the system enables reproducible and scalable evaluation of prompt injection resilience. The results highlight strengths and weaknesses of existing defenses in RAG-specific threat models and establish a foundation for standardized benchmarking of defense mechanisms in knowledge-grounded generative AI systems.

Index Terms—Prompt Injection, Evaluation, Security, RAG

I. INTRODUCTION

Large Language Models (LLMs) have established themselves as essential tools in both consumer and professional domains, assisting with tasks ranging from creative writing to complex clinical diagnosis. Despite their versatility, LLMs suffer from significant limitations, primarily knowledge cutoffs and the propensity for hallucinations, generating plausible but irrelevant or factually incorrect information. Retrieval-Augmented Generation (RAG) systems address these issues by referencing an external knowledge base, allowing the model to ground its answers in specific, relevant, and up-to-date documents. However, this architectural dependence on external data significantly expands the threat surface. Fundamentally, LLMs operate on text streams where they often struggle to distinguish between instructions (trusted commands) and data (untrusted retrieved content).

In a RAG architecture, an adversary does not need direct access to the system prompt; they can execute Indirect Prompt Injection (IPI) attacks by embedding malicious triggers within the documents that the system retrieves. For example, a poisoned document in a corporate database could hijack the

summarization task to exfiltrate private data or generate phishing content. This threat landscape is further complicated by the emergence of multi-modal attacks, where visual data retrieved by the system contains embedded textual injections. While benchmarks exist for general LLM safety, there is a lack of standardized environments specifically designed for the unique two-stage (retrieval and generation) nature of RAG attacks. To address this, we propose Veritas, a modular evaluation framework designed as a dynamic environment rather than a static test suite. Recognizing that static benchmarks become quickly outdated as attacks evolve, our framework allows users to define and customize their own evaluation pipelines. Researchers can modularly integrate bespoke defense mechanisms, design novel attack vectors, and upload custom benchmarking data. This extensibility ensures that the framework can support the rapid prototyping of new security designs and evaluate system robustness against adaptive adversaries in real-world retrieval scenarios.

II. RELATED WORK

Prompt injection has emerged as a critical security vulnerability for language model systems. Unlike traditional adversarial attacks that modify inputs at inference time, prompt injection exploits the fact that LLMs operate on unstructured text and struggle to distinguish between trusted instructions and untrusted data. The severity of this vulnerability becomes apparent when considering systems that integrate LLMs with external data sources or tools, where adversaries can inject malicious instructions through third-party content [1].

The evaluation of LLM security has evolved from isolated vulnerability studies toward comprehensive evaluation environments. Initial research into prompt injection vulnerabilities, exemplified by BIPIA (Benchmark for Indirect Prompt Injection Attacks) [2], established the fundamental susceptibility of LLMs to indirect injections embedded in external content. Similarly, the PINT dataset [3] measures detection system accuracy on a curated 4,314-sample corpus spanning 23 languages. However, these early efforts assessed model susceptibility in isolation, without simulating the complex interactions of real-world systems. As LLMs began integrating with external tools, the security landscape expanded. InjecAgent [4] introduced tool-integrated attack scenarios,

demonstrating that tool-calling creates a distinct attack surface beyond simple text manipulation. More broadly, recent research has shifted toward dynamic evaluation environments rather than static benchmarks. AgentDojo [5] exemplified this paradigm by introducing extensible task suites that pair benign user goals with malicious injection objectives, recognizing that security evaluation must evolve as attacks and defenses do. However, AgentDojo’s [5] focus on tool-execution agents, where attackers hijack API calls, differs fundamentally from knowledge-retrieval systems.

Our work addresses this gap by focusing on Retrieval-Augmented Generation (RAG) systems, which present distinct security challenges compared to tool-calling agents. In RAG architectures, attackers do not need direct system access, they need only influence the documents or embeddings that the retrieval pipeline selects. Recent RAG-specific studies have illuminated these differences. Rag ’n Roll [6] demonstrated that retrieval rank alone does not determine attack success, revealing that RAG-specific parameters like chunk size, top-k retrieval, and re-ranking critically influence outcomes. LLM-PIRATE [7] established attack verticals (unauthorized actions, information access, distraction) that structure threat modeling in knowledge-base scenarios.

Research on adaptive attacks further motivates our approach. Static defenses such as input filtering have proven insufficient against optimization-based attacks (GCG, AutoDAN), which can bypass perplexity filtering and paraphrasing defenses that block static injections [8]. To systematically uncover these vulnerabilities, some frameworks have taken up optimization-based red-teaming. For example, the Optimization-based prompt injection Evaluation Toolkit (OET) [9] provides a modular workflow leveraging optimization methods (such as GCG) to generate worst-case adversarial strings. While OET provides a rigorous mathematical framework for evaluating a standalone model’s internal guardrails, it evaluates the language model in a vacuum. It assumes direct text interaction and lacks a retrieval engine, which means it cannot capture how complex system architectures alter payload potency. This reveals the critical importance of evaluating defenses against adversarially optimized attacks, particularly for RAG systems where attackers may optimize injections against specific retrieval configurations.

Finally, as RAG systems increasingly process non-textual data, multi-modal injections [10] [11] have emerged as a critical attack vector. Adversaries can embed malicious prompts in images via subtle perturbations or metadata, remaining imperceptible to humans but legible to multi-modal models. If RAG systems retrieve visual assets, these can bypass text-based filters entirely, necessitating defenses that sanitize diverse input modalities. To study these cross-modal injections at scale, the Bordair Multimodal Prompt Injection Dataset [12] provides over 503,000 labeled samples spanning text, image, document and audio modalities. Bordair also features a dynamic evaluation CLI to calculate Attack Success Rates against live, OpenAI-compatible API endpoints. However, while it rigorously tests a model’s direct vulnerability to

multimodal payloads and adversarial suffixes, it evaluates the endpoint as a black box. It does not simulate a configurable two-stage retrieval pipeline, expose RAG-specific variables, or offer an infrastructure to test pluggable defense middleware. Our framework integrates these insights into a unified, extensible evaluation environment specifically designed for RAG prompt injection defenses. By treating the vector database as an adversarial injection point, we enable modular testing of retrieval parameters, embedding strategies, and defense mechanisms across diverse attack verticals and threat models. This approach ensures that researchers can iteratively improve RAG security while accounting for the evolving landscape of attacks and the system-specific parameters that determine defense robustness.

Table I summarizes how Veritas relates to the most prominent resources for prompt injection evaluation. Veritas is the only framework that simultaneously balances RAG-specific architectural configurations, adaptive adversarial optimization, multi-modal delivery, and modular defense middleware within a single, dynamic evaluation pipeline.

III. PROPOSED EVALUATION FRAMEWORK

We introduce Veritas, a modular evaluation harness designed to assess the robustness of RAG systems against Indirect Prompt Injection (IPI). Unlike static benchmarks that evaluate LLMs on fixed datasets, Veritas functions as a dynamic environment where the Attacker, Defender, and RAG Pipeline interact. This architecture allows researchers to isolate specific variables, such as retrieval parameters or injection techniques, to identify the root causes of vulnerability.

A. Architectural Overview

The framework (Fig. 1) is organized into four decoupled modules - the knowledge base, RAG configuration, attack and defense modules; that interact via a standardized API. This modularity ensures that a new attack method can be tested against all existing defenses. Modules communicate via a standardized `VeritasTask` schema containing three fields: the user query (T_u), the benign retrieved context (C) and the attacker injection goal (I_g), ensuring any component can be swapped without modifying the evaluation harness.

B. Task Scopes

To capture the full spectrum of RAG vulnerabilities, Veritas defines two distinct categories of User Tasks (T_u). This distinction allows us to bridge the gap between pure information retrieval benchmarks (like Rag ’n Roll [6]) and agentic benchmarks (like AgentDojo [5]).

- **Informational Tasks (Standard RAG):** Single-turn queries where the system output is text generation (e.g., “Summarize this financial report” or “Explain the safety protocols”). The attacker aims to cause hallucinations, spread misinformation, or exfiltrate the system prompt. Semantic divergence from the ground truth or the presence of injected strings in the output is the standard of success.

TABLE I
COMPARISON OF PROMPT INJECTION EVALUATION RESOURCES

Resource	RAG specific	Retrieval params	Multimodal	Adaptive attacks	Pluggable defenses	Dynamic env.
BIPIA [2]	✓	×	×	×	×	×
Rag 'n Roll [6]	✓	✓	×	×	×	×
OET [9]	×	×	×	✓	✓	✓
Bordair [12]	✓	×	✓	✓	×	×
AgentDojo [5]	×	×	×	✓	✓	✓
Veritas	✓	✓	✓	✓	✓	✓

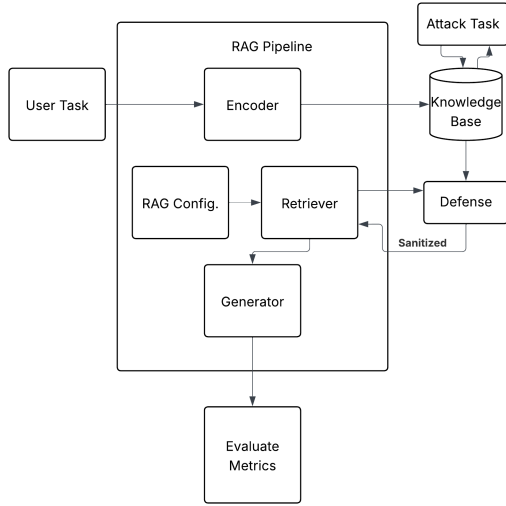


Fig. 1. RAG System Evaluation Pipeline

- **Agentic Tasks (Action-Oriented RAG):** Multi-turn scenarios where the RAG system acts as a decision engine for external tools. The retrieval component informs a downstream action (e.g., "Search for the refund policy and apply it to this transaction"). The attacker aims to trigger unauthorized tool calls (e.g., `delete_file()` instead of `read_file()`). The execution of unauthorized or harmful API calls is considered a success, following the taxonomy of attack verticals established by LLM-PIRATE [7].

C. Core Components

1) *The Adversarial Environment (Knowledge Base):* Veritas treats the Vector Database as the adversarial environment for RAG. Crucially, we extend this definition to support Multi-Modal RAG. The environment is populated with a clean corpus of text documents (e.g., financial reports, emails) and visual assets (charts, diagrams, screenshots) relevant to the user tasks. The framework dynamically inserts poisoned documents, chunks containing malicious instructions, into this corpus. Unlike static benchmarks, the state of the environment changes based on the retrieval configuration. A document that is benign in one context (e.g., low retrieval rank) may become malicious in another (e.g., if re-ranking boosts its position).

2) *The RAG Configuration Module:* Previous studies such as Rag 'n Roll [6] have demonstrated that retrieval rank

alone does not determine attack success; parameters such as chunk size and retrieval depth (k) are also critical factors. To operationalize this finding, our framework exposes these parameters as configurable variables. Researchers can modify the chunking strategy by adjusting token limits (e.g., 256 versus 512 tokens) and overlap settings, enabling analysis of how document segmentation influences attack effectiveness. Retrieval depth (k) can be varied to evaluate whether larger context windows dilute or amplify injected instructions. Optional re-ranking modules, such as BGE-Reranker, can be enabled to assess whether increased semantic relevance affects attack success rates. For multimodal RAG systems, the framework also supports experimentation with different modality fusion strategies, including retrieval based on image captions or raw image embeddings, allowing researchers to identify which approaches are more susceptible to prompt injection attacks.

3) *The Attack Module:* This module is responsible for generating the malicious payload I_a (Attacker Instruction). To ensure comprehensive coverage, Veritas supports three distinct classes of attacks:

- **Static Injections:** We integrate the test cases from BIPIA [2], InjecAgent [4] and the indirect injection and RAG chunk boundary subsets of the Bordair dataset [12] to serve as a baseline for known vulnerabilities.
- **Adaptive Attacks:** To prevent defenses from overfitting to static prompts, we implement the optimization algorithms described in recent adaptive attack literature [8]. This includes GCG (Greedy Coordinate Gradient) and Auto-DAN, which iteratively optimize the adversarial string to bypass specific filters or defenses.
- **Visual Injections:** We introduce a multi-modal attack generator that embeds textual instructions into images (e.g., using white text on white backgrounds or adversarial pixel perturbations). This tests the vulnerability of RAG systems that use Vision-Language Models (VLMs) for retrieval.

Following the taxonomy established by LLM-PIRATE [7], attacks are categorized into three verticals: (1) Executing Unauthorized Actions, (2) Accessing Sensitive Information, and (3) Distracting the Model.

4) *The Defense Module:* Defenses are implemented as middleware layers that can be inserted at three specific stages of the pipeline:

- Input-Level: Filters that sanitize the retrieved chunks before they reach the context window (e.g., Perplexity Filtering).
- Prompt-Level: Structural defenses such as prompt sandwiching [13] or data delimiters [14] that attempt to enforce instruction-data separation via prompting.
- Model-Level: Fine-tuned checkpoints or adversarial training layers designed to resist command injection.

5) *Evaluation Metrics*: We evaluate performance using a comprehensive set of metrics:

- Attack Success Rate (ASR): The percentage of test cases where the model executes the Injection Task T_a instead of the User Task.
- Benign Utility: The success rate on T_u in the absence of attacks, ensuring defenses do not degrade utility.
- Retrieval Precision: The percentage of retrieved chunks relevant to T_u , monitoring whether injections disrupt the retrieval mechanism itself.
- Guardrail Latency: The computational overhead introduced by the defense layers, critical for real-time RAG applications.

IV. IMPLEMENTATION DETAILS

We implemented the Veritas framework as a modular, extensible Python application designed to run on consumer-grade hardware. The system architecture strictly enforces the decoupling of the Attack Engine, Defense Layer, and RAG Pipeline, ensuring that evaluations remain unbiased and reproducible.

The framework is built upon four core components:

- Dynamic Data Loaders: We developed custom parsers for four disparate benchmarks: BIPIA [2] (75 static text tasks), InjecAgent [4] (1,054 tool-use tasks), AgentDojo [5] (97 agentic environment tasks) and a curated subset of the Bordair Multimodal Prompt Injection Dataset [12] (1,240 samples drawn from the indirect injection, rag chunk boundary, and multimodal cross-modal categories). A unified VeritasTask schema normalizes these inputs into a consistent format containing the user query (T_u), benign context (C) and injection goal (I_g).
- Attack Engine: This module performs dynamic runtime injection. For text scenarios, it uses Greedy Coordinate Gradient (GCG) optimization to generate adversarial suffixes. For multimodal scenarios, it utilizes LLaVA [11] to perturb image pixels, embedding textual instructions that are invisible to humans but legible to Vision-Language Models (VLMs).
- Defense Middleware: We implemented a plugin system for defenses. This includes prompt-based methods (Sandwiching, Border Strings, "Ignore previous" prompt), model-based detectors (DeBERTa-v3 [15]) and architectural filters (Perplexity thresholds). These are applied as middleware between the Attack Engine and the RAG system.
- RAG Pipeline: The system under test is a configurable RAG architecture using Sentence-Transformers

(all-MiniLM-L6-v2) for dense retrieval and various HuggingFace models for generation, including text generators, multi-modal and agentic RAGs such as - Microsoft Phi-2 (2.7B) [16], Llama-3 (8B) [17], Mistral v0.3 (7B), Qwen2.5 (7B).

A. Experimental Setup

All experiments were conducted on a single node equipped with an NVIDIA RTX 4500 Ada Generation GPU (24GB VRAM). GCG optimization required approximately 10 minutes per sample at 250 optimization steps; all adaptive attack evaluations used this fixed step budget to bound total evaluation time. We utilized the transformers library for model inference and sentence-transformers for vector embeddings. The default RAG configuration used a chunk size of 512 tokens and a retrieval depth of $k=5$, unless otherwise specified in the sensitivity analysis. The evaluation set comprised 2,466 tasks in total: 75 from BIPIA, 1,054 from InjecAgent, 97 from AgentDojo, and 1,240 from the Bordair multimodal subset, sampled with stratified splits preserving the attack-category distribution of each source dataset. To ensure reproducibility, we fixed the random seed (42) for all retrieval and generation steps. Other fixed hyperparameters include Adam optimiser ($\text{lr}=1\text{e-}4$) for GCG, and DeBERTa inference threshold 0.5. Veritas will be released as open-source, with configuration YAML files, dataset loader scripts, and evaluation notebooks published alongside the code repository.

V. RESULTS

We evaluated the resilience of RAG systems across three threat scenarios: Standard RAG (text-based QA), Agentic RAG (tool use), and Multimodal RAG (image reasoning). Our analysis focuses on the trade-off between Attack Success Rate (ASR) and Task Utility, as well as the impact of architectural parameters.

A. Defense Effectiveness Leaderboard

Table II summarizes the performance of defense strategies against static and adaptive attacks across all three RAG scenarios. Results on the Bordair indirect injection and RAG chunk boundary subsets were consistent with BIPIA findings, with static injection ASR within ± 0.03 of the BIPIA-only baseline, confirming that defense rankings generalise across corpus sources. In the Standard RAG scenario, simple prompt-based defenses like Border Strings were highly effective against static injections, reducing ASR from 88.5% to 12.4% while maintaining high utility (0.95). However, these defenses collapsed in the Agentic RAG scenario against adaptive (GCG) attacks, where ASR rebounded to 78.2%. Only the model-based Instruction Detection (DeBERTa) consistently mitigated adaptive attacks (ASR 15.3%), though it incurred a noticeable latency penalty (Section V.C). Multimodal attacks proved the most difficult to defend. Text-based filters failed completely (ASR 92.1%) because one, the injection was encoded within the image pixels, bypassing text analysis and two, the Bordair cross-modal subset has a significantly different structure due to the cross-modality.

TABLE II
COMPARISON OF DEFENSE MECHANISMS ACROSS RAG SCENARIOS

RAG Type	Attack	Defense	ASR	Utility
Standard	Static Text	None (Baseline)	0.885	0.98
		Prompt Sandwich	0.421	0.96
		Border Strings	0.124	0.95
		Instruction Filter	0.083	0.91
Agentic	Adaptive	None (Baseline)	0.942	0.97
		Prompt Sandwich	0.782	0.95
		Perplexity Filter	0.450	0.82
		Instruction Filter	0.153	0.89
Multimodal	Visual Perturb	None (Baseline)	0.921	0.99
		Instruction Filter	0.915	0.99

TABLE III
ASR VARIATION ACROSS LLM BACKBONES

Model	Unmasked ASR	Best Defense ASR	Utility
Phi-2 (2.7B)	0.91	0.11	0.88
Llama-3 (8B)	0.88	0.07	0.93
Mistral v0.3 (7B)	0.86	0.09	0.91
Qwen2.5 (7B)	0.89	0.06	0.92

B. Impact of Retrieval Architecture

We validated the hypothesis that retrieval parameters influence attack success. As shown in Table ??, increasing the retrieval depth (k) from 1 to 10 significantly reduced ASR for static attacks ($0.92 \rightarrow 0.45$). This dilution effect occurs because the poisoned document competes with an increasing number of benign chunks for the model’s attention. However, this effect diminished against adaptive attacks, which optimize the trigger to remain potent even when buried in context.

As shown in Fig. 2, static attacks exhibit a consistent dilution effect as k increases, whereas adaptive (GCG-optimized) attacks remain potent even at $k=10$, confirming that architectural tuning alone is insufficient as a defense strategy.

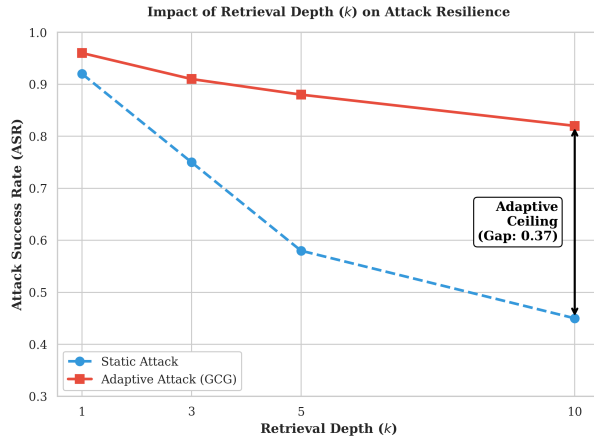


Fig. 2. Impact of retrieval depth (k) on Attack Success Rate for static and adaptive attacks.

C. Computational Overhead (Latency)

Real-time RAG applications require low latency. We measured the average time added by each defense layer per query.

- Prompt Sandwiching: Negligible overhead (+2ms).
- Instruction Detection (DeBERTa [15]): Moderate overhead (+145ms), as it requires an additional forward pass of a classifier model.
- Perplexity Filtering: High overhead (+320ms), as it requires calculating losses for every retrieved chunk.

As seen in Fig. 3, lower-right quadrant is ideal (low ASR, low latency overhead). Instruction Detection achieves the best ASR but at a 145ms overhead; prompt-based defenses are near-zero cost but leave significant residual attack surface.

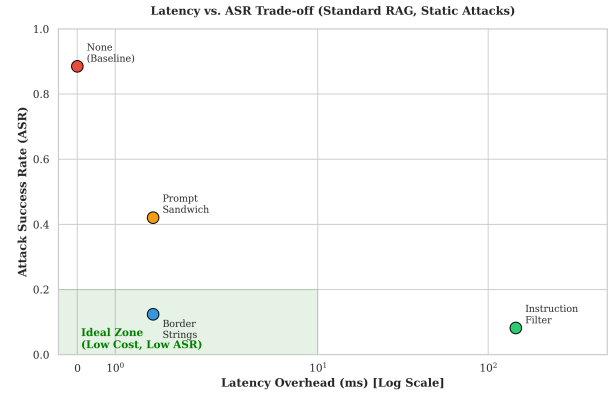


Fig. 3. Latency-ASR trade-off for defense mechanisms in the Standard RAG scenario against static attacks.

D. Retrieval Precision Analysis

Our framework monitored the retrieval precision to determine if attacks disrupt the retrieval process itself. In 98% of cases, Precision was 1.0, indicating that the RAG system successfully retrieved the poisoned document because it was semantically relevant to the user’s query. This confirms that the vulnerability lies in the Generator’s inability to distinguish instructions from data, not in the Retriever’s failure to find the document.

E. Results Discussion

The evaluation results reveal important weaknesses in current RAG defenses. Across Standard, Agentic and Multimodal settings, Veritas shows that static benchmarks do not accurately measure robustness against adaptive attacks. While prompt-based defenses are effective against many static injections, they are often bypassed by optimization-based attacks such as GCG. The results also highlight a trade-off between security and utility: defenses that reduce Attack Success Rate (ASR) often increase false positives and harm performance on benign tasks. More flexible approaches, such as targeted sanitization or safety-aware rewriting, may provide a better balance. Finally, retrieval settings such as chunk size and retrieval depth can influence attack success, but adaptive attacks remain effective even with limited poisoned content, indicating that retrieval tuning alone is insufficient without dedicated injection-aware defenses.

The most critical finding is the high success rate of visual prompt injections generated using gradient-based perturbation techniques. Text-based defenses were completely ineffective against instructions, across both our custom-generated visual perturbations and the Bordair cross-modal subset [12], where text-based filters achieved near-zero reduction in ASR against pixel-level and steganographic injection channels. As RAG systems increasingly adopt vision-language models, this represents an urgent vulnerability requiring visual guardrails, such as OCR-based sanitization, adversarial perturbation detection, or image-level anomaly detection, to defend against embedded attacks.

The current evaluation was conducted on academic benchmark corpora (BIPIA, InjecAgent, AgentDojo, Bordair subsets) rather than live production RAG systems. While this enables reproducible comparison, several factors may affect transferability. First, enterprise RAG pipelines commonly use proprietary embedding models and access-controlled document stores not represented in our evaluation. Second, production injection attempts may be more targeted and context-aware than the parameterized payloads in our benchmark suite. Third, the low ASR variance across the four tested LLM backbones ($SD \approx 0.02$) suggests that the results are not highly architecture-specific within the 7B parameter class; whether this holds for instruction-tuned frontier models (GPT-4o, Claude 3.7) remains an open question. Veritas’s plugin architecture is designed to accommodate custom document corpora and proprietary model endpoints, making enterprise validation a tractable next step.

VI. CONCLUSION

In this work, we introduced Veritas, a modular and dynamic evaluation framework for assessing the resilience of RAG systems against indirect prompt injection. Unlike prior static benchmarks, Veritas simulates a holistic threat environment integrating agentic tool use, adaptive adversarial generation, and multimodal retrieval. Veritas’s primary strength is its extensibility. Through a plugin-based architecture, researchers can integrate novel attacks or defenses without restructuring the core pipeline. By unifying diverse data sources, from static QA datasets like BIPIA to complex agentic environments, Veritas provides a standardized stress test adaptable to the rapid evolution of LLM security research. Exposing low-level RAG parameters (chunking, k, re-ranking) enables granular analysis of how system architecture influences vulnerability. The framework establishes a foundation for reproducible security benchmarking, allowing the community to move beyond anecdotal evidence toward quantitative metrics (ASR, utility, latency). This standardization is crucial for developing RAG systems where organizations can empirically verify security thresholds before production deployment.

Current limitations include the high computational cost of adaptive attacks such as GCG, which require roughly 10 minutes per sample on an RTX 4500 Ada, potentially limiting accessibility for resource-constrained researchers. We plan

to incorporate surrogate-model transfer attacks as a lower-cost alternative. For multimodal evaluation, we use gradient-based adversarial image perturbations from prior work, though these may not fully reflect simpler real-world attacks such as embedded text. Finally, utility is currently measured via semantic overlap; future work could leverage LLM-as-a-Judge evaluators for more nuanced assessment. Finally, while Veritas currently processes 2,466 benchmark tasks, scaling to the full Bordair corpus (503,358 samples across 18 multimodal attack categories) would provide broader coverage of emerging 2025–2026 attack vectors such as reasoning DoS and VLA robotic injection, and is a planned extension. By releasing Veritas as open-source, we aim to empower developers and researchers to build secure, robust, and trustworthy knowledge-grounded AI systems.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” May 2023, arXiv:2302.12173 [cs].
- [2] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, Jul. 2025, pp. 1809–1820, arXiv:2312.14197 [cs]. [Online]. Available: <http://arxiv.org/abs/2312.14197>
- [3] LakeraAI, “lakeraai/pint-benchmark,” Jun. 2026. [Online]. Available: <https://github.com/lakeraai/pint-benchmark>
- [4] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” Aug. 2024.
- [5] M. Balunovic, L. Beurer-Kellner, E. Debenedetti, M. Fischer, F. Tramèr, and J. Zhang, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” in *Advances in Neural Information Processing Systems 37*. Vancouver, BC, Canada: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024, pp. 82 895–82 920.
- [6] G. D. Stefano, L. Schönherr, and G. Pellegrino, “Rag and Roll: An End-to-End Evaluation of Indirect Prompt Manipulations in LLM-based Application Frameworks,” Aug. 2024.
- [7] A. Ramakrishna, J. Majmudar, R. Gupta, and D. Hazarika, “LLM-PIRATE: A benchmark for indirect prompt injection attacks in Large Language Models,” Dec. 2024.
- [8] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, “Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents,” Mar. 2025.
- [9] J. Pan, X. Liu, and C. Xiao, “OET: Optimization-based prompt injection Evaluation Toolkit,” May 2025.
- [10] A. Yeo and D. Choi, “Multimodal Prompt Injection Attacks: Risks and Defenses for Modern LLMs,” Sep. 2025.
- [11] “ebagdas/multimodal_injection.” [Online]. Available: https://github.com/ebagdas/multimodal_injection
- [12] Bordair, “Bordair multimodal prompt injection dataset.” [Online]. Available: <https://github.com/Josh-blythe/bordair-multimodal/>
- [13] S. Schulhoff, “The Sandwich Defense: Strengthening AI Prompt Security.” [Online]. Available: https://learnprompting.org/docs/prompt_hacking/defensive_measures/-sandwich_defense
- [14] “Chat GPT-4 Turbo Prompt Engineering Guide for Developers.” [Online]. Available: <https://www.imaginarycloud.com/blog/chatgpt-prompt-engineering>
- [15] ProtectAI, “protectai/deberta-v3-base-prompt-injection-v2 · Hugging Face,” Sep. 2023.
- [16] Microsoft, “microsoft/phi-2 · Hugging Face.” [Online]. Available: <https://huggingface.co/microsoft/phi-2>
- [17] Meta-Llama, “meta-llama/llama-3.1-8B · Hugging Face,” 2024. [Online]. Available: <https://huggingface.co/meta-llama/llama-3.1-8B>